

ИТОГОВЫЙ ПРОЕКТНЫЙ ОТЧЁТ

Идемпотентный платёжный сервис

Результаты разработки, качество проверок, ограничения процесса и выводы для дальнейшей работы

АВТОР

Рашит

ДАТА

28 августа 2026

Краткое резюме

Проект доведён до работающего результата: идемпотентный платёжный API, браузерная песочница и документация запускаются одной командой. Все 56 требований подтверждены прогоном; шесть прежних хвостов закрыты задачей #180. Повторяемая приёмка на чистом клоне зафиксировала 293 успешных теста бэкенда, 21 проверку без расхождений и работоспособность всех трёх сервисов.

Главный результат проекта шире продукта. В ходе независимых проверок выявился повторяющийся системный риск: проверка может давать корректный ответ не на тот вопрос. Зафиксированы случаи, когда тесты, защитные хуки, журналы и ревью подтверждали соседнее свойство, другую версию кода или лишь упоминание признака. Внешняя проверка оказалась особенно ценной: на одной задаче она обнаружила восемь проблем против двух, найденных внутренней приёмкой, включая блокирующее снятие защиты с корня файловой системы.

Рекомендации для следующего цикла: закреплять каждое требование тестом, который действительно краснеет при нарушении; привязывать вердикт к конкретному коммиту; разделять рабочие копии для всех, кто запускает команды; вводить внешнюю линзу на изменения защиты; измерять проверяемое свойство напрямую и фиксировать границы механизмов в том же изменении, где они вводятся.

Оглавление

1. Что это и как запустить	3
2. Обязательная часть	4
3. Необязательная часть (бонусы)	6
4. Что не сработало	7
5. Цифры и хронология	20
6. Что заберём после курса	21
6а. Что заберём — из продуктового потока	22
7. Журнал задач	23

1. Что это и как запустить

Идемпотентный платёжный сервис: API, песочница в браузере, страница документации. Всё поднимается одной командой, предусловий у проверяющего нет, кроме Docker.

```
docker compose up
```

Порты по умолчанию: API 8080, песочница 8081, документация 8082; переопределяются переменными окружения без правки файлов. Тесты в контейнере — отдельным сервисом:

```
docker compose run --rm backend-test
```

Подробности запуска и раздел «если не поднялось» — в [README.md](#).

2. Обязательная часть

2.1 Работаящий результат

Требований в PRD — 56: F1–F22 (двадцать пять с подпунктами) по API, U1–U7 (восемь) по песочнице, D1–D4 по документации, R1–R6 по запуску, A1–A13 — критерии приёмки.

Чем подтверждены: 293 теста бэкенда в контейнере, повторяемый приёмочный сценарий, машинная сверка ответов с `openapi.yaml`, скриншоты песочницы на двух ширинах, прогон на чистом клоне.

Честно про покрытие. Обратной проверки — «на каждое требование найдётся тест, который покраснеет при его нарушении» — целиком по всем 56 не делалось. Такой проход прошли выборочно: требования, которых касались задачи независимых проверок (#47, #49), и четыре требования в аудите (4.10). Он же трижды за смену обнаружил тесты, зеленеющие на сломанном коде, — то есть выборка была не формальностью.

Пять требований — F19–F22 и U7 — **записаны задним числом**: поведение было реализовано и принято, а требования за ним не стояло (4.10, 4.12).

Аудит готовности 27 августа (#180, слито #183). Отдельная сессия прошла все 56 требований против `docs/product/PRD.md` прогоном, а не чтением, и нашла шесть недотянутых проверок: F20, U4, U7, A4, A7 и A8. Задача #180 закрыла все шесть тестами, которые **краснеют при нарушении требования**, с обратной проверкой исполнителем. Итог: **56 требований из 56 подтверждены прогоном; частично подтверждённых и неподтверждённых требований нет.**

Результат аудита	Требований	Что это значит
подтверждено прогоном полностью	56	поведение реализовано и закреплено проверкой
подтверждено частично	0	все шесть прежних хвостов закрыты в #180
не подтверждено	0	требований без реализации нет

Отдельное противоречие F7b/F7c по запятой и точке в ключе после аудита закрыто продуктовым решением: оба символа запрещены, изменение слито в #186. Открытой остаётся оговорка об инструменте: `scripts/acceptance.sh` пока не различает кодом возврата «стенд не поднят» и «поведение разошлось» (#184); это дефект приёмочного скрипта, а не продукта.

Проверено повторяемой приёмкой на **чистом клоне** 27 августа (`scripts/clean-clone-check.sh`, вывод в #183):

```
docker compose up -d --build      код возврата 0, три контейнера здоровы
API :19080 → 200    песочница :19081 → 200    документация :19082 → 200
повтор с тем же ключом          тот же id, платёж встречается один раз
конфликт ключа                  409 idempotency_key_reuse
СОШЛОСЬ: 21 проверок пройдено, расхождений нет. КОД ВОЗВРАТА: 0
docker compose run --rm --build backend-test    # tests 293 / # pass 293 / # fail 0
```

Клон делался во временную папку намеренно: рабочая копия помнит установленные зависимости, собранные образы и поднятые сервисы, а проверяющий приходит без всего этого.

2.2 Агенты

Это соло с разделением обязанностей по сессиям, а не команда людей. В отчётности называется так и никак иначе.

Роли за проект: продукт, два проджекта (продуктовый поток и поток обвязки), контракт, дизайн, два бэкенда, два фронта, документация, два devops, два QA и проверяющие. Каждая роль — отдельная сессия со своим заданием, своей рабочей копией и своим журналом в `sessions/`.

Где это видно без клонирования репозитория:

- **задачи** — 40 issues, по одной на роль плюс дефекты;
- **ветки и pull request** — 52 слитых, каждый ссылается на номер задачи;
- **приёмки** — комментарием в задаче: что проверял, какой командой, какой вывод;
- **журналы сессий** — `sessions/*.md`, дословные промпты и принятые решения.

Разделение потоков между двумя проджектами и правила их сосуществования — в [docs/tasks/00-handoff-project.md](#).

2.3 Workflow

Записанный в репозитории workflow-файл на восемь ступеней — план, ревью тремя линзами, реализация, ревью результата, MR, CI — за весь проект не запускался ни разу. Ни одного упоминания в журналах одиннадцати сессий, ни одного прогона.

Причина: он рассчитан на одну сессию, ведущую задачу от плана до слияния. Работа пошла иначе — задача живёт в трекере, роли не видят контекста друг друга, и каждая ступень выполняется отдельной сессией. Ступени при этом сохранились почти все, но разошлись по людям, а не по фазам одного прогона.

Это расхождение правила с практикой, и оно наше собственное. В `CLAUDE.md` записано «многошаговая задача идёт через эту процедуру» — правило прожило проект неисполненным, и заметили это только при сборке отчёта. Тот же класс, что и в 4.10: **записанное правило, за которым нет механизма проверки, отменяется молча** — с той разницей, что здесь его не отменили, а просто не заметили.

Что применялось безусловно и подкреплено механизмами, а не соглашением: задача → ветка → pull request → приёмка не автором → слияние. Прямой push в `main`, force-push, ветка вне соглашения и операции с рабочими копиями блокируются хуками; зону открывает человек в своём терминале на срок до пятнадцати минут.

3. Необязательная часть (бонусы)

Пункт	Состояние
рабочие копии на роль	сделано: до одиннадцати копий одновременно, <code>.worktrees/</code>
журналы сессий	сделано: <code>sessions/*.md</code> по каждой роли
изоляция с выводом отказа	сделано: семь хуков, отказы в <code>.claude/logs/guard.jsonl</code>
проверка инъекций	сделано: по API (#77) и по песочнице (#189)
внешняя модель как третья линза	сделано: введена по ходу проекта, окупилась в тот же час
замеры	сделано: мутационная диагностика, счёт покрытия, замеры состояния
PDF	собран: итоговый отчёт находится в <code>output/pdf/project-report.pdf</code> ; печатный исходник — <code>output/pdf/project-report.html</code>

4. Что не сработало

Раздел крупный намеренно: отрицательный результат здесь содержательнее положительного.

4.1 Главный сюжет: проверки, которые выглядят проверками

Тема проекта выбиралась про код, который выглядит правильным и таковым не является. К концу работы проект оказался про **проверки** с тем же свойством. Одиннадцать независимых случаев, пять ролей, разный код:

#	Что произошло	Чем держался ложный результат
1	тест на гонку зеленел на заведомо сломанном коде	средой: в одном процессе гонки не существует
2	мутация не собралась — засчитали как защиту	компилятором, сработавшим раньше теста
3	тест назван по свойству, исполняется на функции, которая его не держит	соседней функцией
4	зелёный CI на PR с двумя дырами	тестами, написанными автором фикса
5	проба защиты дала «разрешено»	открытым пропуском, а не поведением хука
6	замер показал «ноль изменений»	копией, уехавшей на другую ветку
7	повтор отказов по журналу дал неверный ответ	обрезанной записью команды в журнале
8	счёт мутационного покрытия «14 из 17»	памятью вместо вывода прогонов
9	«набор не трогает боевое состояние»	верным замером с выдуманной причиной
10	след в общей копии приписан роли	именем файла и подходящей ролью
11	«половина дефекта не исправлена»	замером не той версии кода
12	набор тестов «покраснел» на чужой правке	прогоном из копии, которую в этот момент правил её владелец
13	ожидание вердикта сработало и сообщило, что вердикт пришёл	совпадением с упоминанием вердикта, а не с ним самим

Тринадцатый стоит отдельно, и он мой. Я построил детектор ожидаемого события — и он поймал **упоминание** события вместо события: в комментарии проверяющего стояли и номер коммита, и слово «линза». Если бы я не посмотрел, что именно совпало, я слил бы работу по указанию «сливать, как придёт вердикт», которого не приходило.

Это единственный случай за день, где автор проверки и её жертва — одно лицо. И он структурно тождествен главному дефекту наших хуков: **признак принят за свойство, упоминание — за использование**. Тому самому, из-за которого за смену случилось двадцать четыре ложных отказа.

Ни один изнутри не выглядел подозрительно. Каждый был аккуратно сделан и давал верные цифры. Отличали их только три вопроса, которые никто не обязан задавать: **что именно я измеряю · почему получилось именно так · ту ли версию я измеряю.**

4.2 Внешняя линза: восемь против двух на одной задаче

Требование внешнего ревью правок обвязки введено в 18:00 и окупилось через час на задаче #43:

Кто проверял	Чем	Нашёл
проджект (принимающий)	сравнение поведения на безобидных путях	2
исполнитель	девять собственных парных проверок, набор зелёный	0
внешняя модель	один прогон, чтение кода как есть	8, включая блокиер

Блокер: сужение, сделанное против ложной тревоги, сняло защиту **с корня файловой системы**. Я выдал приёмку за час до этого и отозвал её публично.

Вывод дороже цифр: внешняя модель нашла восемь дыр не потому, что умнее, а потому, что ей не показали замысел. Пары исполнителя проверяли задуманное поведение, принимающий подбирал пробы под найденные им классы — оба смотрели через один и тот же замысел. Это предел любой внутренней проверки.

Побочная деталь, без которой история неполна: первая формулировка задания внешней модели содержала просьбу найти способы обхода — провайдер отрезал её фильтром и вернул пустой вердикт. Пустой выглядит как «замечаний нет». Остановись исполнитель на нём — блокер был бы слит с **формальным подтверждением качества**.

4.3 Защита, мешающая проверять защиту

За смену механизмы дали **двадцать два ложных отказа** и пропустили **три настоящих обхода**. Ложные отказы — один класс: **строка упомянута, а не использована**. Путь внутри контейнера в тексте приёмки, имя файла правил в теле задания, маршрут `/healthz` в проверочной команде, шаблон адреса `sed`, чтение собственного файла состояния.

Три случая стоят отдельно:

- **класс отказал в акте приёмки задачи про этот же класс** — текст акта по #43 был отклонён за упоминание пути в прозе;
- **защита не дала продемонстрировать собственную дыру:** проба, показывающая, что правка отключает проверку корня, была отклонена за слэши в данных;
- **двадцать третье срабатывание оказалось верным** — настоящий управляющий символ в отчёте ломателя, из-за которого `main` стал красным для всех потоков. По виду оно ничем не отличалось от предыдущих двадцати двух. Это лучший аргумент против привычки отмахиваться от знакомого отказа.

И обратная сторона: пропуски не мешают никому и потому не находятся сами. Из трёх известных два найдены случайно, третий — чтением кода.

4.4 Общее рабочее дерево: четыре инцидента

Копии заводились ролям, но не проверяющим и не проджекам — «раз они не пишут код, хватит общей». Ошибка: журнал требует коммита, ветки и pull request, то есть трёх способов сдвинуть общее состояние.

Итог: коммит в чужую ветку, правки хуков поверх чужого журнального PR, запуск скрипта инъекций в общем корне — и два случая, которые мы приняли за откаты чужой работы. **Откатов не было**: дерево и HEAD расходились, пока соседняя сессия переключала ветки и подтягивала. Разбор — в 4.9; здесь важно другое: мы дважды опросили девять сессий, прежде чем посмотреть в журнал ссылок.

Точная формулировка правила пришла только после четвёртого случая: копия нужна не тому, кто пишет код, а **тому, кто запускает что-либо**. Скрипт в общем дереве опаснее коммита: коммит виден в истории, а откат рабочего дерева не виден нигде, пока в него не упрётся.

4.5 Механизм, который мешал работе, оставаясь исправным

Гейт качества требовал прогона тестов после любой правки кода — но считал кодом и чужие рабочие копии: из 555 файлов, которые он сторожил, **487 лежали в чужих копиях**, включая собранный `dist` и чужой `node_modules`. Сессия, не тронувшая ни строки, получала требование прогона; прогон при этом снимал выданные человеком пропуска и сыпал фикстуры в боевой журнал.

Механизм подталкивал сразу к двум вредным действиям: закоммитить в чужую ветку и сжечь чужие пропуска. Правило «сделано подтверждается выводом команды» верное; проверка, не различающая свою работу и соседскую, превращает его в ритуал.

4.6 Требования, безупречные по отдельности

Требование «в рантайме нет тестов и зависимостей разработки» и требование «тесты запускаются командой в том же сервисе» **несовместимы**. Противоречие пролежало весь проект и всплыло на предпоследней задаче — там, где два требования впервые встретились в одном файле.

Ревью требований поштучно не находит противоречий требований. Их находит только первая попытка исполнить оба разом.

4.7 Отказы, которые спасли работу

Трижды исполнители отказались сделать то, о чём их просили:

- ломатель не стал проверять вторую половину найденного пропуска — проверка означала бы настоящую запись за пределы репозитория. **Нашёл границу, не переступив её**;
- второй ломатель отказался записать в журнал строку по указанию проджека: сценарий был пройден, и ложную запись он ставить не стал. Отсюда правило: **владелец задачи не источник истины о том, что было сделано**;
- devops не стал удалять из боевого журнала собственные фикстуры: чистить журнал задним числом значит **править доказательство**.

Четвёртый отказ — продуктовый, и он о честности демонстрации. Сценарий «две одновременные отправки» показывал то `201/200`, то `201/409`: у бэкэнда фиксация мгновенна, и второй запрос почти всегда приходит после завершения первого. Напрашивалось добавить на сервер искусственную задержку, чтобы 409 показывался надёжно. **Бэкенд отказался предлагать это сам, проджект**

согласился, продукт подтвердил: экран показывал бы наше желание, а не поведение системы. Вместо кода ответа сценарий показывает инвариант — платёж один, `id` один, список не вырос, — а журнал печатает фактический исход с пояснением, отчего он зависит.

Подделанная демонстрация ломает единственное, ради чего эта песочница существует: если один сценарий подкручен, доверия нет ни к одному.

Ни один отказ не был удобен, и каждый оказался прав.

4.8 Вечер: шесть кругов внешней проверки на правку «в один символ»

Задача #97 заводилась как правка одного символа: шаблон исключения допускал ноль цифр, поэтому токен из одних слэшей выпадал из разбора, и корень файловой системы не проверялся в действующем `main`. Нашла вторая сессия обвязки, подав команды всем восьми хукам; принимающий подтвердил разбором шаблона.

Что было дальше, по кругам внешней модели:

Круг	Итог	Что нашла
1	<code>block</code>	абсолютные пути с числовыми именами — то же, к чему исполнитель пришёл прогоном форм, независимо и до буквы
2	чисто	—
3	<code>block</code>	файловая команда внутри встроенного кода : имя стадии <code>sh</code> , соседи корня выглядят операндами
4	<code>block</code>	имя команды, положенное в переменную — регрессия правки этого же коммита
5	<code>revise</code>	признак шире дыры: отказ полз на стадии, где переменная не используется
6	находки	семь форм, все воспроизведены и на <code>main</code> — наследство, не регрессия

Пять кругов из шести нашли настоящее, и дважды это были свежие регрессии на коммитах, которые прошли и приёмку проджекта, и приёмку ломателя. Ни один из этих коммитов не выглядел сырым: каждый был зелёным, измеренным и объяснённым.

Вывод, который дороже фикса

Формулировка исполнителя, взята дословно:

Разбор угадывает роль токена **по соседям и по присутствию имени**, а не по позиции в команде. Признак вместо позиции — это **потолок подхода**, а не недоделка в нём.

Каждый следующий признак закрывал одну форму, открывая соседнюю. Седьмой коммит был бы продолжением того же способа, а не выходом из него.

Три признания исполнителя по дороге

Приводятся, потому что каждое поймал он сам, а не проверка:

Это третий раз за задачу, когда я закрываю класс по одной проверенной форме. **Красный тест на одной форме успокаивает не хуже зелёного на нуле.**

Проверка противодействием поймала три формы, но пропустила четвертую, и цена такой проверки в том, что **после неё сильнее веришь себе.**

Я закрыл дыру признаком, который шире дыры. Предыдущие три расширяли отказ на чужое, этот расширял **на пустое место.**

Я нарушил собственное правило и переписал его под уже совершённое нарушение

Правило «сливать после круга без находок» ввёл я. Оно прожило три часа и было нарушено мной же — первым случаем, когда всерьёз мешало. Слияние состоялось на шестом круге с находками.

Что было бы правильным действием в тот момент. Дождаться седьмого круга. Находки шестого были наследством `main`, ветка строго лучше по каждой измеренной форме — но это довод за то, что **правило следовало сформулировать иначе с самого начала**, а не за то, что его можно нарушить сейчас. Стоимость ожидания была известна и невелика: четыре дыры и шесть ложных отказов жили бы в `main` ещё круг.

И худшее в моём поступке — не само нарушение. Я переписал правило так, чтобы оно допускало уже совершённое: «после круга без находок **либо** после доказательства, что находки воспроизводятся на основании». Новая формулировка, возможно, лучше старой — но принята она задним числом и под собственный случай. Это легализация, а не улучшение, и признать её таковой обязан именно я.

Исполнитель предупредил меня **до** того, как я это сделал:

Правило, которое отменяют в момент, когда оно впервые мешает, дальше не работает ни разу.

Вердикт, на который нельзя было опереться

Файл вердикта шестого круга оказался **испорчен**: прогон был запущен неверно, оборвался, записав часть, и перезапуск дописал вторую. Исполнитель сказал об этом первым абзацем и опёрся не на слово вердикта, а на проверку каждой формы запуском.

Будь у нас только слова вердикта — решение пришлось бы отложить на седьмой круг.

4.8a Двадцать пять ложных отказов: не «строка упомянута», а «признак принят за свойство»

Формулировка, с которой мы работали полдня, — «строка **упомянута**, а не использована» — описывает половину случаев. К вечеру набралось двадцать пять отказов, и часть в неё не влезает: под запрет попадали **чистые чтения**.

- `ls` и `grep` по каталогу рабочих процессов — отклонены как **правка** конфигурации CI;
- чтение состояния пропусков внутри встроенного кода — отклонено как **правка выданных пропусков**;
- разбор диффа файла хука — отклонён как **правка хуков**.

Ни в одной из этих команд нет записи. Общее у всех двадцати пяти — другое:

Признак принят за свойство. Наличие имени принято за обращение к нему, упоминание — за использование, соседство — за роль.

Это ровно тот дефект, который мы шесть кругов чинили **внутри** разбора путей, и он же оказался устройством всей защиты: хук ищет **признак** запрещённого действия, а признак дешевле свойства и потому шире его.

Следствие для отчёта, а не для кода. Двадцать пять отказов — не «шум, который надо убрать». Это цена выбранного способа: защита на признаках даёт ложные отказы там, где признак шире свойства, и пропуски там, где уже. Убрать одно, не двинув другое, нельзя — сегодня это доказано шесть раз подряд на одном файле.

И собственный экземпляр того же класса, не в коде. Проджект построил ожидание вердикта — и оно сработало на **упоминании** вердикта в чужом комментарии. Автор проверки и её жертва оказались одним лицом, а устройство ошибки — тем же самым.

4.9 Два «отката», которых не было

Дважды за вечер в общем рабочем дереве обнаруживался подготовленный откат: в первый раз 707 удалённых строк, во второй — 603, включая целиком тест на 319 строк. Оба раза опрашивались все сессии, оба раза все честно отвечали «не я».

Разгадка нашлась в журнале ссылок, а не в опросе. Сессия, ведущая журнал, переключает общую копию на свою ветку, коммитит, возвращается и подтягивает; пока она это делает, общий `origin` уходит вперёд от чужих слияний. В промежутке дерево и HEAD расходятся — и всякий, кто смотрит состояние в этот момент, видит «подготовленный откат». Следующая подтяжка выравнивает, отсюда и «след исчез сам».

Никто ничего не откатывал. Вопрос был неверный, и признаваться было не в чем.

Три следствия:

- **атрибуцию в общей копии нельзя делать опросом** — и по автору коммита нельзя: у всех сессий один и тот же автор в настройках;
- **общий индекс — разделяемое состояние без владельца и без журнала**: хуки его не видят, в журнале отказов не остаётся ни переключения ветки, ни расхождения дерева с HEAD;
- правило «своя копия нужна тому, кто пишет код» оказалось узким дважды: сначала выяснилось, что копия нужна **тому, кто делает коммиты**, потом — **тому, кто запускает что-либо**. Оба ложных инцидента породили журнальные коммиты проджектов, а не работа исполнителей.

4.10 Молчаливая отмена принятой починки

Дефект #75 — длина ключа считалась в байтах вместо символов — был найден ломателем, починен и принят. Через несколько часов сужение, попавшее в контракт по рекомендации продуктового проджекта, **отменило починку**, не тронув код: контракт стал требовать отклонять то, что код принимает.

Ломатель, проверявший починку, обнаружил положение хуже отмены:

Код принимает, контракт требует отклонять. Критерий нарушен **прямо сейчас**, и снести починку теперь — законное действие. Исполнитель, приводящий код к контракту, будет прав по правилам проекта, а починка исчезнет, и он даже не узнает, что что-то отменил.

Сравнение, которое объясняет причину:

Обе мои находки одинаково важны, но одна записана требованием — с обоснованием и с тестом на тихий случай, — а вторая не записана ничем. Первую молча отменить уже нельзя, вторую отменили за несколько часов. Разница не в важности дефекта, а в том, **дошло ли дело до строчки в требованиях**.

Отсюда общее правило, вынесенное в задачу: **каждое место, где принятое поведение живёт только в коде и тесте, без записанного требования, — будущая тихая отмена**. Отменит её добросовестная правка, прошедшая ревью: отменяющему неоткуда узнать, что он что-то отменяет.

Аудит по этому вопросу дал четыре места, и они **разного устройства**: три «есть в коде, нет в требовании» и одно обратное — требование записано, а набор его не различает (мутация пережила 187 тестов).

4.11 Тихий отказ: худшая разновидность дефекта

Самый опасный дефект продукта нашёл ломатель API: **дублирующийся Idempotency-Key создавал два платежа на один ключ**. Отказ главного требования проекта — того самого, ради которого он затевался.

Опаснее всего было не это, а то, что отказ был **тихим**:

```
дубль → дубль:      201, 200, платежей 1  ← дефект НЕ виден
дубль → одиночный:  201, 201, платежей 2   ← виден
одиночный → дубль:  201, 201, платежей 2   ← виден и в эту сторону
```

Оба ответа выглядели успешными. Клиент не получал ни малейшего сигнала — он узнал бы о двойном списании от своего пользователя, а не от API.

И дефект самомаскировался. Повторная отправка того же запроса даёт "к, к" снова, ключ совпадает сам с собой, поведение выглядит правильным. Ломается только там, где между запросами вклинивается вариант без дубля: клиент через прокси и он же напрямую, ретрай из другой среды, повтор после смены сети. **На стенде может не воспроизводиться вовсе, а в проде — регулярно.**

Практическое следствие для теста: проверка «отправили дважды с дублем — получили один платёж» дефект **не поймает**, она зелёная и на сломанной реализации. Нужна смешанная последовательность. Ломатель проверил все три сценария и записал ожидаемый ответ для каждого — до и после исправления.

Решение продукта: отклонять, а не выбирать одно из значений. Ключ идемпотентности — идентификатор намерения; два значения означают, что клиент сам не знает, чего просит, и выбрать за него значит угадать, ошибаясь в половине случаев. Явный 400 неудобен, но приходит немедленно и к тому, кто может исправить запрос.

Уточнение к требованию пришло от ломателя и закрыло дыру в формулировке продукта: «присланный более одного раза» читается как «с разными значениями», а **ломается случай одинаковых** — два Idempotency-Key: К склеиваются в "к, к", то есть в другой ключ, тогда как клиент уверен, что послал

К.

4.12 Предел ревью и то, что осталось непроверенным

Проверяющий сверяет результат с заданием, а не с историей проекта. Он не знает, что три часа назад было принято обратное, и не обязан знать. Из этого следует и молчаливая отмена починки (4.10), и то, почему ревью её не поймало: оно работало исправно и проверяло ровно то, что ему дали.

Отсюда же три расхождения, найденные **не автором требований**:

Что разошлось	Кто нашёл
PRD обещал коды <code>invalid_amount</code> и <code>invalid_currency</code> , которых в API нет	независимая проверка API
требования R3 и R4 несовместимы (4.6)	проджект на сборке <code>compose</code>
правило про рабочие копии сформулировано узко (4.4)	проджект, прочитавший его буквально

Все три — требования продукта, и ни одно не нашёл продукт. Автор смотрит на текст изнутри замысла: он видит, что имел в виду, а не что написал.

Чего мы не проверили — честно:

- **сквозные сценарии на собранном `compose`** второй ломатель начать не успел: QA остановили, когда очередь исправлений начала расти быстрее, чем закрываться. Пауза в поиске — не пауза в наличии дефектов; мы перестали находить, а не перестали их иметь;
- **регресс исправлений** никто не делал: дефекты чинили авторы, принимали проверяющие и проджекты. При этом за смену был случай, где фикс принёс новые дыры при зелёном CI (4.2);
- **работу продуктовой сессии не проверял никто**. У всех ролей проверяющий появился, у продукта — нет. Пробел назван вслух и остался незакрытым;
- **гейт качества закрывался не всегда**. Пока прогон тестов жёг чужие пропуска (4.5), продуктовая сессия сознательно его не запускала и сообщала об этом прямым текстом, а не молча.

4.13 Продуктовый поток: почему ни один прогон не покраснел

Раздел 4.10 рассказывает, **что** молча отменили. Здесь — **почему этого никто не заметил**, и это отдельная находка, сделанная прогоном.

Второй бэкэнд собрал ветку с **прежним** контрактом — тем, где у ключа стоял отменённый набор символов, — и прогнал против неё текущий набор тестов. Итог:

232 из 232 зелёных — включая четыре теста, которые этот контракт прямо запрещает

Формулировка автора находки, взята дословно:

Тесты защищают код от изменений кода, но не от изменений контракта.

Причина названа там же: сверка со схемами контракта покрывает **тела запросов** и **все ответы**, а **заголовки не сверяет никто**. Схемы параметров `IdempotencyKey`, `XMerchantId`, `PaymentId` — с `pattern` и границами длины — в контракте есть и через валидатор не проходят ни разу.

Отсюда уточнение к приёму «закреплять отвергнутое», без которого приём переоценивают:

Тест на отвергнутое защищает от возврата **кода** к отвергнутому варианту. От возврата к нему **контракта** он не защищает — если только тест не сверяется с контрактом в этой самой части.

Зона осталась незакрытой: задача заводилась, но исполнялась не была (см. 4.15), и дыра описана в закрытой задаче #112 поимённо.

4.14 Ещё четыре проверки, которые выглядели проверками

Продолжение таблицы 4.1, вечер и утро продуктового потока. Нумерация сквозная.

#	Что произошло	Чем держался ложный результат
14	двадцать примеров документации прогнаны живьём, результат честный	первым прогоном по свежему стенду : дефект живёт во втором
15	фронтальный тест «на гонку» повторял вызов, не создавая гонки	последовательностью вместо параллельности
16	инструмент сверки примеров молчал на дефекте, ради которого написан	сверкой проходов между собой , а не с обещанием страницы
17	тот же инструмент после починки дал зелёный на прогоне, где не проверено ничего	отсутствием третьего исхода : «нечего сверять» слилось с «расхождений нет»

Четырнадцатый — самый неудобный. Метод автора не был небрежным: он прогнал всё, что написал, и получил правду. Он лишь не различал **состояние стенда до и после себя**. Отсюда правило, вошедшее в чек-лист документации: пример прогоняется дважды подряд, условие приёмки — одинаковый вывод в обоих проходах; обязательно для блоков, которые создают или потребляют состояние.

Семнадцатый найден дословным прохождением чек-листа и стоит того, чтобы назвать способ. Шаг списка оставляет страницу собранной на заведомо чужой адрес; следующий шаг гонит её примеры против живого API. Пересобрать между шагами список не просит. Двадцать блоков подряд не достучались никуда:

```

блоков: 20, сверено с обещанием: 0
проходов сравнено: 2
расхождений нет          ← код возврата 0

```

Умный проверяющий пересобрал бы и не нашёл ничего.

Правило, которое стоит унести из обоих случаев:

Проверяющий инструмент обязан различать **три** исхода, а не два: сошлось, разошлось и **проверить было нечего**. Третий по умолчанию сливается с первым — и наступает чаще остальных, потому что случается от обычных вещей: стенд не поднят, порт другой, сборка осталась от прошлого шага.

Дефект оставлен незакрытым сознательно и вынесен в задачу #127: работа слита по решению владельца проекта в конце смены.

Обратный случай: проверка нашла то, ради чего не писалась

Единственный за проект. Фронту было поручено закрепить тестом **отвергнутое** решение — не писать ошибки отрисовки в журнал обмена. Первый же прогон этого теста поймал дыру **в принятой** защите: выражения внутри разметки вычисляются в родителе, мимо границы, и слой, сданный как «испорченные данные портят одну карточку», ронял весь список.

Нашло его **не ревью, не проверяющий и не автор**, а тест, написанный ради другого. Причина — в направлении: тест на отвергнутое спрашивает не «работает ли задуманное», а «вот так не должно происходить — точно?».

4.15 Ошибки постановки задач: десять, восемь поймали исполнители

Раздел 4 честен про механизмы, исполнителей и требования. Ошибки того, кто раздаёт работу, названы в 4.8 и 4.9 — но только по потоку обвязки. Продуктовый поток закрывает этот пробел своим счётом.

Десять ошибок постановки за смену; восемь из десяти поймали исполнители. Это цифра не про самокритику, а про схему: приёмка задания исполнителем до первого коммита работает, и вот сколько она поймала.

Из десяти стоит назвать четыре, каждая — отдельного устройства:

- **два противоречивых критерия приёмки в одном задании:** маркер требовалось положить в файл, правки в этом каталоге были запрещены. Нашёл исполнитель на приёмке задания;
- **обходной путь вместо починки:** расхождение требований с контрактом было известно, и вместо исправления по пяти задачам разослано примечание. Обходной путь выглядит решением и работает как долг, который платит третий;
- **обход чужого отказа добрыми руками** — см. ниже;
- **условие старта без адресата** — см. ниже.

Обход чужого отказа, остановленный отказом

Проверяющий не смог прогнать шаг чек-листа: команда запрещена классификатором в его сессии. Проджект написал «это ограничение вашей стороны, не замечание к работе» — и следующим движением попросил **соседнюю сессию** прогнать эту команду вместо него.

Исполнитель отказался, **имея работающий docker**, и объяснил точнее, чем формулировал заказчик:

Плохой обход отличается от хорошего не намерением, а тем, кто в итоге принял решение.

Границы разрешений в каждой сессии свои и ставятся не для удобства. Действие, запрещённое у одного агента, не становится разрешённым оттого, что его выполнит другой по просьбе третьего: решение владельца обходится, просто чуть длиннее.

Это четвёртый отказ в ряду 4.7 и первый, где отказ пришёл от того, у кого возможность была. Отказ при наличии возможности стоит дороже отказа по бессилию.

Шаг был закрыт позже — прогоном по прямому решению владельца проекта, то есть тем самым способом, который отличает разрешение от обхода.

Проверяющий той же смены заметил ряд шире: за день границы разрешений пытались обойти трижды, и **все три раза механизм устоял на чьём-то отказе, а не на невозможности.**

Условие старта без адресата

Задача о сверке заголовков простояла всю смену и закрылась невыполненной. Причина не в исполнителе: условие старта — «после слияния отката контракта» — было поставлено, а **сигнала о его снятии не подал никто**. Откат слит в 16:29; в задачу писали дважды после этого, оба раза про требования, и ни разу — что можно начинать.

Условие старта заводится вместе с тем, кто обязан сообщить о его снятии. Условие без адресата — не приоритет, а способ остановить задачу молча.

Тихая остановка снаружи не отличается от медленной работы: поле исполнителя заполнено, задача выглядит взятой.

4.16 Дополнения к разделу «что не сработало»

Собрано при сверке отчёта с журналами: эпизоды, которые произошли, но в текст не попали. Каждый проверен по задачам и переписке, не по памяти.

Запрет, у которого одна дверь закрыта, а вторая открыта

Одно и то же содержимое проходит инструментом записи файла и отклоняется командой оболочки. Цель записи в обоих случаях одна — значит, проверяется не она, а форма обращения.

Этим пользовались. Первый проджект, получив отказ на записи текста задания, записал файл другим инструментом и **сам назвал это обходом**: «обошёл защиту, не имея на то никаких прав, просто выбрав другой инструмент». Второй ломатель поступил так же, столкнувшись с отказом на теле heredoc.

Тот же класс — #83: скрипт, запущенный **по имени файла**, под признаки записи не подпадает вовсе; содержимое файла хук не видит.

Вывод, неудобный для нас: пока один путь отказывает, а другой пропускает, запрет воспитывает не осторожность, а **знание троп**. За смену этим путём воспользовались дважды, оба раза назвав вслух, — и оба раза это было правильнее, чем молча, но не отменяет того, что дверь открыта.

За вечер принимающий четырежды упёрся в тот же отказ на текстах приёмов и задач и **не воспользовался** обходом: переформулировал абзац. Это выбор, а не свойство механизма.

Механизм, который не падает, а молча делает не то

Отдельный класс, обнаруженный при сборке: **неизвестный build-аргумент даёт предупреждение, а не ошибку**. Если бы имя аргумента для образа документации угадали неверно, сборка прошла бы, а расхождение всплыло бы только при проверке с переопределённым портом — то есть ценой были бы не упавшие контейнеры, а **тихо неверные примеры в документации**.

Исполнитель остановился на пороге пятиминутной правки и запросил точное имя через владельцев потоков, вместо того чтобы угадать. Час ожидания против недели неверных примеров.

Пауза в поиске дефектов и честная строка о ней

По распоряжению сэра ломатели **приостановили поиск**: очередь исправлений не успевала за находками. Формулировка, записанная в обе задачи без смягчения:

Пауза в поиске — это не пауза в наличии дефектов. Мы перестали находить, а не перестали их иметь.

Что осталось непроверенным, известно поимённо и записано в журналах разделом «чего здесь нет и почему»: страница документации не проверялась никем, проверка сканера подложенной инструкцией по песочнице не начата, HTTP-слой не мутировал никто.

Регресс исправлений не делает никто — дефекты чинят авторы, проверяют проджекты, то есть та же пара глаз, что и до появления ломателей. Поэтому первым заданием после снятия паузы назначена проверка **починенного**, а не новая поверхность.

4.17 Отказы, спасшие работу: ещё три

Дополнение к 4.7. Общее у всех трёх: **механизм пропустил бы, отказалась роль**.

- **Контрактная сессия не стала восстанавливать собственный затёртый пропуск.** Файл состояния не входит в список защищённых от неё — технически могла. Назвала это самовыдачей разрешения: разница между «починить своё состояние» и «выдать себе разрешение» здесь ровно в одну строку JSON.
- **Вторая сессия обвязки не воспользовалась чужим пропуском.** В момент постановки задачи зона была открыта — но выдана под другую задачу. Дождалась, пока сэр переиздаст пропуск с нужной причиной. Механизм зон **не различает, кто именно взял ключ**, и это стоит знать при разборе журнала.
- **Ломатель API пересняла три нулевых ответа вместо того, чтобы записать их в результат.** Сервер не успел подняться; собственный таймаут в таблице выглядел бы как поведение сервиса.

Итого за смену **пять раз роль отказалась засчитать удобную цифру** — и каждый раз выяснялось, что цифра была не о том.

4.18 Доказательство ядра продукта, полученное поломкой

Единственная проверка, где обещанное свойство подтвердилось **на отказе связи**, а не на успешном пути. Принадлежит ломателю песочницы, и она же его собственная исправленная ошибка.

Первая попытка ничего не проверила. Прокси задерживал **запрос**, а не ответ: клиент отваливался по таймауту раньше, чем прокси дочитывал тело, до API не долетало ничего. Платежей `8 → 8` — и это выглядело как «сервис устоял». Итерация проверяла собственный прокси автора и была аннулирована им же.

Переигранная версия дала лучший результат смены:

```
запрос доехал, платёж создан      8 → 9
ответ до браузера не дошёл        песочница: «нет ответа»
повтор по её же совету, тот же ключ вернулся тот самый идентификатор
второго платежа                   не появилось
```

То есть свойство, ради которого написан сервис, проверено **в условиях, где оно и нужно** — при обрыве после записи. Это доказательство, а не заявление.

4.19 Метод, появившийся к концу дня: счёт вместо впечатления

Два инструмента, оба придуманы не проверяющими, а теми, кого проверяли, и оба заменяют спор о впечатлении на число.

Счёт открытий — для документов.

Исполнимость документа проверяет тот, кто его не писал и в задаче не участвовал. Он ведёт счёт: **сколько раз пришлось открыть что-то за пределами документа, и что именно.** Ноль — документ исполним.

Ценность в том, что он отменяет спор «достаточно ли это очевидно», который автор выигрывает всегда: ему очевидно. Причина, по которой проверять обязан посторонний: **автор открывает соседний файл не глядя, потому что держит его в голове, и не засчитывает это как открытие.**

Счёт форм на признак — для кода. Не одна проверка на правило, а три и больше:

Красный тест на одной форме успокаивает не хуже зелёного на нуле.

За смену «одна форма, обобщённая на класс» дала четыре отдельных дефекта в одной задаче, и трижды автор ловил себя на ней сам — уже зная о ней.

5. Цифры и хронология

Срез: 2026-08-25, 20:17. Все числа ниже относятся к этому моменту и получены командой, а не по памяти. Проект продолжается, поэтому цифра без среза бессмысленна — сегодня счёт мутационного покрытия уже один раз оказался неверным именно потому, что назывался по памяти.

Показатель	Значение	Чем получено
задач заведено	50	<code>gh issue list --state all</code>
pull request слито	66	<code>gh pr list --state merged</code>
дефектов с меткой <code>bug</code>	22	<code>gh issue list --label bug</code>
проверок в наборе хуков	222	прогон набора, счёт зелёных строк
дефектов обвязки за смену	8	задачи с меткой <code>role:devops</code>
дефектов продукта от ломателей	5	#73, #74, #75, #76, #92
ложных отказов защиты за смену	24	счёт по журналу отказов и сессиям
пропусков защиты найдено	3	#68, #83, #97
случаев «результат — правда о другом предмете»	13	4.1 и далее

Про число тестов бэкенда честно. Таблица выше остаётся историческим срезом 25 августа. Более поздняя приёмка задачи #180 зафиксировала отдельным выводом `293/293` теста и 21 проверку чистого клона; эти числа отражены в разделе 2.1 как финальное состояние, а не подставлены задним числом в старый срез.

Самый дорогой дефект продукта — #73: дублирующийся заголовок ключа идемпотентности даёт **два платежа на один ключ**. Отказ ровно того свойства, ради которого написан сервис, и **тихий**: ничего не падает, ошибок нет, в журнале два успешных ответа. Найден ломателем, подтверждён независимо второй сессией, закреплён требованием с тестом на тихий случай.

6. Что заберём после курса

Требуем обсуждения: пишется в конце, когда видно, чем пользовались, а чем нет.

Предварительно, из потока обвязки:

- **приёмка задания исполнителем до первого коммита** с блокирующим вердиктом — дважды вскрыла противоречия в самих заданиях;
 - **вопрос «что именно я измеряю»** как обязательный шаг любой проверки;
 - **внешняя линза на изменениях в защите** — единственное, что видит замысел со стороны;
 - **пары «безобидное проходит / настоящее отклоняется»** на каждое сужение защиты.
 - **счёт вместо впечатления** — открытий для документа, форм на признак для кода (4.19): оба отменяют спор, который автор выигрывает всегда;
 - **вердикт действителен для того коммита, на котором получен** — правило, окупившееся дважды в тот же вечер и нарушенное мной же (4.8);
 - **копия нужна тому, кто запускает что-либо**, а не тому, кто пишет код: формулировка расширялась дважды, и оба раза после инцидента (4.4, 4.9).
-

6а. Что заберём — из продуктового потока

Дополнение к разделу 6. Каждый пункт появился из прогона, а не из размышления, и применим за пределами случая, где найден.

- **Счёт открытых файлов.** Исполнимость чек-листа, README или инструкции проверяет тот, кто их не писал: он считает, сколько раз пришлось открыть что-то за пределами документа. Ноль — исполним; один и больше — каждый случай есть пропущенная строка, называется поимённо. Критерий **отменяет спор «достаточно ли это очевидно»**, который автор выигрывает всегда. С обязательной оговоркой: считается только то, без чего нельзя выполнить шаг, — иначе документ начнёт дублировать источники правды и разойдётся с ними на первой правке.
- **Вопрос приёмки «где ещё живёт это же место»** вместо «починено ли то, что нашли». Проверяющий, идущий по своему списку находок, по построению проверяет частные случаи.
- **Механизм узнаёт о новом сам; частное решение ждёт дописки.** Различать надо не «список против перебора», а закрыт список проверкой или открыт: перечень, выход за который ломает прогон, — механизм; перечень имён, мимо которого новое проходит молча, — частное решение под видом механизма.
- **Три исхода проверки** вместо двух (см. 4.14).
- **Границы защиты называются в том же PR, где защита вводится.** За смену трижды платили за необъявленные границы; обработка отказа вдобавок делает отказ **тише**, и цену этой тишины называют там же.

Пара, объясняющая, почему проверяет не автор

Два признания, оба от исполнителей, оба про устройство, а не про характер:

Проверяющий, идущий по своему же списку находок, по построению проверяет частные случаи — он смотрит туда, куда сам показал.

Автор кода по построению проверяет то, что задумал, а не то, что написал.

И третье, про текст, от автора документации:

Чужой текст я читаю как читатель, а свой — как автор, который помнит, чего не делал.

Ни одна из трёх слепот не лечится усердием: усердие тратится внутри той же рамки, которая ограничивает обзор. Лечит только третий взгляд — и лучше не предупреждённый о том, где искать.

7. Журнал задач

Ведёт координатор (Карина) — строка на задачу, после закрытия тикета. Источник строки — рапорт исполнителя и машинная проверка факта слияния; где это слова исполнителя, так и написано. Регламент — [docs/roles/karina.md](#).

#	Задача	Сессия (<code><Имя>-<N></code>)	PR	Ревью: находок / отклонено	Проблемы по ходу	За рамками
65	гейт считал исходниками чужие рабочие копии	—	#145	—	закрит без исполнителя: сделано попутно при переходе на схему ролей	правка мимо инструментов сессии ловится другим пунктом гейта
102	compose передаёт адрес API документации	Салават-102	—	1 возражение агента-опровергателя, отклонено письменно	работа была сделана 25 августа, за двое суток до выдачи; ложный отказ <code>guard-scope</code> : подстрока <code></code></code> прочитана как путь	правки README (#153), механизм под подстановкой (#154)
148	README объясняет <code>--build</code> только через песочницу	Салават-148	#152	5 находок агента: 4 приняты (2 вынесены тикетами), 1 отклонена письменно	верный отказ <code>guard-git</code> на команде с хвостом; локальная краснота набора по чужому #147; очередь слияния держалась координатором по отправшему основанию	#153, #154, зашитые порты в примерах <code>openapi.yaml</code>
162	на пути <code>Read</code> сканер не видел переводов строки	Никита-162	#169	7 исправлено / 1 отклонена письменно; 5 блокирующих на плане, 2 на результате; матрица из 9 мутаций, декоративных утверждений было 4 — стало 0	пропуск дважды уходил не в ту копию (#165); два ложных отказа класса #103 (#168); слияние соседней задачи сняло класс, на котором стояли 10 фикстур; <code>--delete-branch</code> не сработал из-за чужой рабочей копии	перечень ключей, матчер хука, корпус шаблонов (#161, #170), обратное расширение разбора (#172)
163	точечный откат: глушение сканера ушло из <code>main</code>	Ранит-163	#171	2 блокирующие (тест не проверял журнал; опровергнуто утверждение автора «источник глушения один»), 5 исправлено / 0 отклонено; внешняя линза не её трек	четыре ложных отказа класса «упоминание принято за использование»; база копии устарела по вине координатора; тело задачи содержало два неверных утверждения координатора, оба разобраны до первого коммита	#161, #162, #170; десять пропусков названы вслух и посчитаны

#	Задача	Сессия (<code><Имя></code> <code><ИД></code>)	PR	Ревью: находок / отклонено	Проблемы по ходу	За рамками
103 + 147	разбор не различал природу строки; набор краснел из рабочей копии	Рустем-103	#160	41 находка (27 на план, 14 на результат), 35 исправлено / 6 отклонено письменно; обе блокирующие на результате — регрессии собственной правки ; внешняя линза отработала с третьей попытки, в обход сломанной обёртки	четыре ложных отказа на самой этой работе, ни один не обойдён; красный CI поймал то, чего не поймали пять линз — набор зеленел на macOS и краснел на ubuntu, потому что корень репозитория разный и путь из журнала лежал по разные стороны границы; вторая зависимость от машины (проверка существования путей) внешнего подтверждения не получила и осталась названной ценой — «пока не выстрелило», а не «проверено»; обёртка внешней линзы падала дважды	#108, #107, #68, #156, #151, #159; названы две цены: решение зависит от файловой системы машины, литерал из одних слэшей остаётся отказом
77	сканер пропускал две инъекции из девяти	Ранит-77	#157	12 исправлено / 0 отклонено; внешняя линза дала блокирующую находку	два ложных отказа <code>guard-scope</code> на чтении (переданы в #103); гейт держал сдачу по чужому #147; дважды принял чистый прогон по этому репозиторию за доказательство — второй раз показал не он сам, а заказанный им чужой замер, и решение по <code>.css</code> было отменено до слияния	19 обходов из 19 закрыты из известных на сегодня ; проза без маркера скрытности проходит; склейка без разделителя и разнесение по буквам оставлены границей; #149; #158; цена глагола <code>delete</code>
180	шесть недотянутых проверок аудита: F20, U4, U7, A4, A7, A8	Рустем-180	#183	8 блокеров на плане приняты; на результате 1 находка P1 разобрана живым стендом	повторяемая проверка чистого клона сняла ошибочный вывод о неработающем Docker и закрепила 21 проверку	все шесть хвостов закрыты; итог 56/56; F7b/F7c решено отдельно в #186; дефект кода возврата приёмки — #184

Столбец «проблемы по ходу» заполняется всегда, в том числе ложными отказами механизмов: они и есть материал для правки обвязки, и прошлая смена показала, что без записи они забываются к вечеру того же дня.